

SkyGuard AI: Meteorological Telemetry Ingestion, Fault Detection, and Sensor Health Governance Platform

Technical Specification, Pipeline Manual & Demonstration Guide

1. Executive Summary

SkyGuard AI is a meteorological telemetry ingestion, fault detection, and sensor health governance platform. It is engineered to monitor automated weather station (AWS) networks for sensor degradation, physical noise spikes, frozen transducers, calibration drift, and environmental anomalies.

The platform divides responsibilities across two complementary stages of the data-quality pipeline: 1. **ESP32 Edge Module**: Evaluates incoming local sensor signals at the physical station site to provide immediate local hardware protection for locally certifiable edge faults (`CERTAIN_FAULT`) and maintain station-level resilience during network disruptions. 2. **Central SkyGuard Engine**: Ingests raw telemetry streams to perform spatio-temporal ML scoring, cross-station spatial peer corroboration, diurnal uncertainty budget estimation, and SHAP explainability.

Pipeline Architecture Overview

- **Sensor Site (ESP32 Edge Engine)**: Physical Sensor Array (DHT22 / BME280 / Transducers) → ESP32 Microcontroller (240MHz Dual Core, 320KB RAM) → Level 1 Edge Safety Rules (Physical Bounds Check, Instant Step-Spike Filter, Dropout & Fail-Low Protection) → Emergency Local Actuation / Relay & USB-Serial / Wi-Fi Telemetry Ingestion Packet.
- **Central Cloud / Server System**: FastAPI Central Ingestion Service (`POST /api/ingest/observation`) → State Manager & Dynamic Window → 28 Neighboring Station Peer Buffers → Decision Engine & Fault Classifier (Isolation Forest ML, Diurnal Uncertainty Budget, Cross-Channel Mahalanobis) → TimescaleDB / Mirror CSV Store & Interactive Operations Dashboard (Live Trends, SHAP Risk Cards).

2. System Architecture: ESP32 Edge & Central SkyGuard Integration

The ESP32 edge module and central SkyGuard engine operate at complementary stages of the data-quality pipeline:

- **ESP32 Edge Module**: Operates directly at the physical station site to provide immediate local protection for faults that can be certified from local evidence (such as physical bounds violations, transducer short-circuits, or signal dropouts) and maintains station-level operational resilience during network disruptions.
- **Central SkyGuard Engine**: Operates at the network server level to process continuous telemetry streams, performing spatio-temporal detection, multi-station spatial peer corroboration, diurnal uncertainty baseline evaluation, and SHAP explainability.

By integrating on-device local validation with central spatio-temporal analytics, the system delivers reliable monitoring both at the physical station site and across the broader weather network.

2.1 Scope of the ESP32 Edge System

The ESP32 module focuses on local signal validation and station-level resilience:

1. **Immediate Local Fault Protection**
2. Sensor short-circuits, severe voltage spikes, or out-of-range physical readings are certified instantly from single-station physical bounds.
3. The ESP32 evaluates these local bounds directly on-device, enabling immediate local hardware protection (triggering safety relays or local alerts) before transmitting data.

4. Station-Level Resilience During Network Disruptions

5. Severe weather events can disrupt cellular or Wi-Fi links.
6. Under network outages, the ESP32 maintains continuous local logging and basic bounds checking, preventing blind spots at the physical station until central connectivity is restored.

7. Payload Tagging & Traffic Optimization

8. Raw observations are tagged with on-device edge status (`SAFE_FORWARD` , `DEFER_TO_CENTRAL` , `CERTAIN_FAULT`) to provide immediate metadata context upon arrival at central ingestion.

2.2 Scope of the Central SkyGuard Engine

Anomalies requiring broader temporal or multi-station context are evaluated by the Central SkyGuard Engine:

- **Spatial Peer Corroboration:** Distinguishing an isolated sensor fault from a genuine regional weather front requires real-time spatial covariance matching across adjacent weather stations.
- **Temporal & Diurnal Context:** Detecting gradual sensor drift or frozen transducer states requires evaluating 24-hour diurnal historical expectations, CUSUM accumulation, and multi-channel Mahalanobis distances over rolling memory windows.
- **Authoritative System Verdict:** The Central SkyGuard Engine maintains final control over overall anomaly verdicts, risk scores, parameter reconstruction suggestions, and SHAP feature attribution.

2.3 Pipeline Functional Responsibility Matrix

Data-Quality Pipeline Stage	ESP32 Edge Engine	Central SkyGuard Engine
Primary Scope	Immediate Local Protection & Resilience	Temporal, Spatial & Multivariate Context
Physical Range Bounds Check	Certified from Local Evidence	Enforcement & Archival
Instant Sensor Dropout Filter	Certified from Local Evidence	Record & Track
Sub-ADC Step Spike Filter	On-Device Differential Check	Diurnal Uncertainty Budget
Station Network Outage Survival	Local Buffering & Resilience	Requires Active Connection
Spatial Peer Corroboration	Excluded (Single Node)	28-Station Spatial Covariance
CUSUM Drift & Frozen Value Detection	Local Stuck-Output Check	Diurnal Volatility & Multi-Hour Check
Isolation Forest ML Model	MCU Memory Constraint	Trained Ensemble Model
SHAP Feature Attribution	Excluded from MCU	Full Interactive Explainability
Final System Verdict Authority	Edge Advisory Tag	Authoritative Central Verdict

3. Firmware Flashing & ESP32 Manual

3.1 Repository Directory Structure

The ESP32 source code is located in the `EDGE/` directory:

```

HELL-T0-SKY/
├── EDGE/
│   ├── esp32/                                <-- PlatformIO / ESP-IDF C++ Project (C++17)
│   │   ├── src/
│   │   │   ├── main.cpp                      <-- Firmware Main Entry & Dual-Core Task Loop
│   │   │   └── edge/
│   │   │       ├── edge_engine.cpp           <-- Level 1 Inference Rules Engine
│   │   │       └── edge_engine.h             <-- Struct Definitions & Thresholds
│   │   └── platformio.ini                    <-- PlatformIO Environment Configuration
│   └── esp32_arduino/                        <-- Arduino IDE C++ Sketch (Single-File Flashing)
│       └── edge_engine.cpp                    <-- Standalone Arduino Firmware

```

3.2 Method A: Flashing via Arduino IDE

1. **Install Arduino IDE:** Download and open Arduino IDE (v2.x recommended).
2. **Install ESP32 Board Support:**
3. Go to **File** -> **Preferences**.
4. Add to *Additional Boards Manager URLs*:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
5. Open **Tools** -> **Board** -> **Boards Manager**, search for **esp32** by Espressif, and click **Install**.
6. **Install Required Libraries:**
7. Go to **Tools** -> **Manage Libraries...**
8. Install **ArduinoJson** (v6.x or v7.x by Benoit Blanchon).
9. Install **DHT sensor library** (by Adafruit) if using physical DHT22 sensors.
10. **Open Firmware File:**
11. Open file: `EDGE/esp32_arduino/edge_engine.cpp`.
12. **Connect Hardware & Select Board:**
13. Connect ESP32 DevKit board via USB-C or Micro-USB cable.
14. Select Board: **Tools** -> **Board** -> **ESP32 Arduino** -> **ESP32 Dev Module**.
15. Select Port: **Tools** -> **Port** -> **COM3** (Windows) or **/dev/ttyUSB0** (Linux/Mac).
16. **Flash Firmware:**
17. Click **Upload** (->).
18. Open **Serial Monitor** at **115200 baud** to view firmware boot logs:

```

SkyGuard AI – ESP32 Level 1 Firmware Active
Continuous-Time Causal Edge AI Engine • Zero-Leakage

```

```

[Boot] ESP32 dual-core initialized @ 240MHz.
[Edge AI Engine] Level 1 safety rules online.

```

3.3 Method B: Flashing via PlatformIO

1. Open VS Code with **PlatformIO Extension** installed.
2. Open Project Folder: `EDGE/esp32`.
3. Connect ESP32 via USB.
4. Upload Firmware: `bash pio run --target upload`

5. Monitor Serial Output: `bash pio device monitor --baud 115200`

4. Demonstration & Testing Guide

Follow these steps to run the end-to-end telemetry system.

Execution Workflow Sequence

1. **Operator:** Starts FastAPI Server (port 8000) & opens Operations Center Dashboard (`http://localhost:5173`).
 2. **Hardware Streamer Script:** Transmits observation payload to physical ESP32 microcontroller over USB Serial (`python scripts/test_esp32_hardware.py --port COM5`).
 3. **ESP32 Edge Microcontroller:** Evaluates Level 1 Local Edge Advisory (`SAFE_FORWARD` , `DEFER_TO_CENTRAL` , or `CERTAIN_FAULT`).
 4. **Backend Server:** Ingests payload via `POST /api/ingest/observation` , computes spatio-temporal ML & diurnal uncertainty score, and broadcasts WebSocket update (`TELEMETRY_TICK`).
 5. **Frontend Dashboard:** Live UI updates packet counts, interactive trend lines, and SHAP risk cards.
-

Step 1: Start Central Backend System

Open a terminal in the root project directory:

```
# 1. Activate Python Environment (Python 3.10+)
.\venv\Scripts\Activate.ps1

# 2. Start FastAPI Central Server
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Verification: Open `http://localhost:8000/docs` to inspect the OpenAPI documentation.

Step 2: Start Frontend Dashboard

Open a second terminal window:

```
cd FRONTEND
npm run dev
```

Verification: Open `http://localhost:5173` in your browser.

Step 3: Stream ESP32 Telemetry

Connect ESP32 via USB and execute the test runner:

```
# Stream dataset telemetry through physical ESP32 microcontroller over USB Serial
python scripts/test_esp32_hardware.py --port COM5 --interval 2.0 --csv data_esp32/AWS-CHN-024.csv
```

Note for testing without physical ESP32 hardware:

You can run the virtual hardware node simulation: `powershell python scripts/virtual_esp32_node.py --csv data/AWS-CHN-024.csv --interval 1.0 --rows 50`

4.1 Steps to Reproduce ESP32 Benchmarks

To independently verify all empirical metrics and fault detection scorecards, execute the following commands from the project root directory:

1. Run Dedicated ESP32 Certain Fault Benchmark Suite

Evaluates all 60,480 telemetry observations across 28 Indian weather stations in `data_esp32/`:

```
python scripts/benchmark_esp32_entry_protection.py
```

Expected Output Summary: - **Evaluated Telemetry Observations:** 60,480 - **True Positives (TP - Certain Faults):** 5,192 - **False Positives (FP - False Alarms):** 0 - **False Negatives (FN - Central Deferred):** 607 (deferred to Level 2 Central Engine) - **ESP32 CERTAIN_FAULT Precision:** 100% (Zero False Alarms, \$FP = 0\$) - **False Positive Rate (FPR):** 0%

2. Run Complete Automated System Test Suite

Executes unit, integration, and contract tests across backend services, edge rules, and telemetry parsers:

```
python -m pytest tests/
```

Expected Output Summary: **67 / 67 Passed (100%).**

5. System Performance & Empirical Benchmarking

5.1 ESP32 Edge Certain Fault Scope Benchmark (28 Station Datasets)

Evaluated across **60,480 continuous telemetry observations** across 28 automated weather station datasets in India (Bhopal, Chennai, Delhi, Kolkata, Mumbai, Ranchi, Varanasi):

Benchmark Metric	Empirical Value	Operational Interpretation
Total Evaluated Telemetry Observations	60,480	Full 28-station continuous telemetry dataset
True Positives (TP - Certain Faults)	5,192	On-device certifiable edge faults intercepted
False Positives (FP - False Alarms)	0	Zero false alarms on clean operational weather
False Negatives (FN - Central Deferred)	607	Non-certain anomalies deferred to Level 2 Central Engine
ESP32 CERTAIN_FAULT Precision	100%	100% precision on certified edge fault scope (\$FP = 0\$)
False Positive Rate (FPR)	0%	Zero false alarm rate on clean weather stream
Overall Network Fault Recall	Low (By Design)	ESP32 local recall is low against full network anomalies; complex spatio-temporal faults are deferred to Central Engine
Automated System Test Suite	67 / 67 Passed	100% test suite pass rate

5.2 Breakdown Across ESP32 Certifiable Fault Categories

Certifiable Edge Fault Scope Category	True Positives (TP)	False Positives (FP)	Scope Status
1. Sensor Disconnection / Acquisition Failure (ADC / NaN Dropout)	278	0	Certified Edge Fault
2. Invalid or Sentinel Readings (Electrical Rail Floor / Fail-Low)	596	0	Certified Edge Fault
3. Physically Impossible Values / Hard Range Violations	651	0	Certified Edge Fault
4. Sensor Saturation / Rail Clipping (0% / 100% RH)	0	0	Deferred to Central Level 2
5. Missing Samples / Heartbeat Failure (>3h Gap)	0	0	Certified Edge Fault
6. Timestamp / RTC Integrity Failures (Clock Rollback)	0	0	Certified Edge Fault
7. Clear Communication / Transmission Frame Failure	0	0	Certified Edge Fault
8. Established Sensor Freeze / Stuck Output (\$\ge 3\$ Identical Samples)	3,667	0	Certified Edge Fault
TOTAL POOLED EDGE EVALUATION	5,192	0	100% Precision (Zero FP)

6. Telemetry Data Payloads

6.1 ESP32 Ingestion Payload (`POST /api/ingest/observation`)

```
{
  "event_id": "evt_7f8a9b0c1d2e",
  "station_id": "AWS-CHN-024",
  "device_id": "esp32-devkit-v1-serial",
  "observed_at": "2026-09-28T14:35:00.000Z",
  "sequence_number": 9,
  "readings": {
    "temperature_c": 24.0,
    "pressure_hpa": 1011.1,
    "humidity_pct": 74.0
  },
  "edge_inference": {
    "edge_status": "SAFE_FORWARD",
    "is_anomaly": false,
    "tier_fired": 5,
    "anomaly_type": "none",
    "confidence_llr": 0.10
  }
}
```

6.2 Central Decision Response & WebSocket Tick (`TELEMETRY_TICK`)

```
{
  "type": "TELEMETRY_TICK",
  "station_id": "AWS-CHN-024",
  "timestamp": "2026-09-28T14:35:00.000Z",
  "mode": "edge",
  "reading": {
    "temperature_c": 24.0,
    "pressure_hpa": 1011.1,
    "humidity_pct": 74.0
  },
  "verdict": {
    "is_anomaly": false,
    "anomaly_score_pct": 5.0,
    "model_confidence_pct": 95.0,
    "rule_confidence_pct": 0.0,
    "fault_type": null,
    "severity": "low",
    "health_status": "HEALTHY",
    "suggested_values": {
      "temperature_c": null,
      "pressure_hpa": null,
      "humidity_pct": null
    },
    "decision_basis": "Level 2 Central ML & Diurnal Uncertainty Baseline Normal",
    "likely_faulty_sensors": []
  },
  "edge_inference": {
    "edge_status": "SAFE_FORWARD",
    "tier_fired": 5
  },
  "ingest_time_ms": 1759082100123
}
```

7. Troubleshooting Guide

Issue / Symptom	Cause	Resolution Command
<code>SerialException: Could not open port COM5</code>	Port occupied or incorrect COM port	List ports: <code>python -c "import serial.tools.list_ports; print([p.device for p in serial.tools.list_ports.comports()])"</code>
<code>FastAPI 404 Station AWS-CHN-024 not registered</code>	History store cleared	Execute <code>POST /api/admin/clear-history</code> or restart FastAPI.
ESP32 Terminal reads <code>WiFi connection failed</code>	Wi-Fi credentials unconfigured	ESP32 automatically uses USB Serial Host Bridge mode; data streaming continues over USB serial.
Benchmark / test failures	Local cache mismatch	Execute <code>python scripts/benchmark_esp32_entry_protection.py</code> or <code>python -m pytest tests/</code> .